codigo.py

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 9s >

Este arquivo é malware (dropper/loader malicioso).

Análise passo a passo

1. **Ofuscação pesada (técnica clássica de malware)**

   O código original usa:

   - Nomes de variáveis aleatórios sem sentido.
   - `__import__('base64')` e `__import__('zlib')` em vez de imports diretos.
   - Uma string base64 longa comprimida com zlib.
   - XOR com a chave fixa `134` sobre o payload descomprimido.
   - `exec(compile(...))` no final para executar o código deofuscado.

   Isso é feito deliberadamente para dificultar a análise estática e a detecção por antivírus.

2. **Payload deofuscado**

   Após decodificar (base64 → zlib → XOR 134), o código revela um script Python completo (~437 linhas) com comportamento claramente malicioso:

   - **C2 via blockchain Solana**: Consulta repetidamente a API RPC da Solana (`getSignaturesForAddress`) no endereço fixo `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` . Extrai o campo `memo` de transações, limpa e faz `json.loads()` para obter a configuração/C2 (incluindo um link base64).
   - **Download e execução de Node.js**: Baixa a versão oficial do Node.js (v22.9.0) para o diretório home do usuário, extrai e executa.
   - **Execução de payload JavaScript remoto**:
     - Faz GET em uma URL obtida do C2.
     - Lê headers ofuscados (`ivbase64` e `secretkey`).
     - Escreve um arquivo temporário `i.js` contendo:

```JavaScript
var https = require("https");
const secretKey = '...';
const _iv = Buffer.from('.
eval(atob('...'));  // pay
```

Copied link to clipboard. Shared links can be viewed by anyone with the link.

    - Executa o Node.js em background (`subprocess.Popen` com `CREATE_NO_WINDOW` no Windows e stdout/stderr redirecionados para `/dev/null`).
- **Evasão**:
    - Aguarda 10 segundos antes de agir.
    - Detecta sistema em russo (locale, timezone, idioma, username) e **aborta** a execução nesses casos (típico de malware que evita análise em laboratórios russos ou de pesquisadores de língua russa).
    - Usa arquivo `~/init.json` com timestamp para limitar a frequência de execução (cooldown de 2 dias).
    - Múltiplos endpoints RPC da Solana para resiliência.

3. **Comportamento típico de malware**
    - Dropper/loader multiestágio.
    - C2 “stealth” via blockchain (difícil de bloquear).
    - Download de interpretador legítimo (Node.js) para executar código arbitrário remoto via `eval(atob(...))` .
    - Execução silenciosa e persistência parcial via arquivo de controle.
    - Técnicas de ofuscação + anti-análise.

Conclusão

Não é benigno.

É um **malware loader/dropper** que usa a blockchain Solana como canal de comando e controle, baixa Node.js e executa um payload JavaScript remoto ofuscado. O objetivo final do payload JS (não presente no arquivo) tipicamente é stealer, ransomware, botnet, minerador ou acesso remoto, mas a infraestrutura e o comportamento do loader já são suficientes para classificá-lo como malware.

Recomendação: Não execute este arquivo em nenhuma circunstância. Trate-o como amostra maliciosa.

Qual é a categoria principal deste malware?

Worked for 2s >

Categoria principal: Loader (Downloader / Stage-1 Loader)

Justificativa

O malware analisado não realiza por si só o roubo de dados, criptografia de arquivos, mineração ou controle remoto permanente. Sua função principal é:

1. Contatar o C2 (via memos de transações
2. Baixar um interpretador legítimo (Node.js)

Copied link to clipboard. Shared links can be viewed by anyone with the link.

3. Baixar um payload remoto (código JavaScript ofuscado)
4. Executá-lo silenciosamente em segundo plano (`eval(atob(...))`)

Esse comportamento é típico de um **loader** (também chamado de downloader ou dropper de segundo estágio). Ele serve apenas como estágio inicial (stage-1) para entregar e executar o malware real (stage-2), cujo conteúdo depende do que o atacante colocar no payload JavaScript remoto.

Classificações secundárias comuns

- **Trojan.Downloader** / **Trojan.Loader** (classificação clássica de antivírus)
- Pode evoluir para **Stealer**, **RAT**, **Cryptominer** ou **Ransomware**, dependendo do payload final entregue via o C2.

↘ Qual é o malware principal

↘ Como o payload final é executado?

↘ Como evitar execução de scripts remotos

Copied link to clipboard. Shared links can be viewed by anyone with the link.